AlgorithmicTypography — Cheat Sheet

SETUP & LIFECYCLE

Minimal sketch — auto-render mode

```
import algorithmic.typography.*;

AlgorithmicTypography at;

void setup() {
  size(800, 800);
  at = new AlgorithmicTypography(this);
  at.loadConfiguration("config.json");
  at.initialize();
}

void draw() {
  // auto-renders — nothing to add
}
```

Programmatic config (no JSON file)

```
Configuration cfg = new Configuration.Builder()
  .initialTiles(12, 12)
  .changedTiles(6, 6)
  .changeTime(5000)
  .character("@")
  .build();

at.setConfiguration(cfg);
at.initialize();
```

Restart & keyboard

```
void keyPressed() {
  if (key == 'r') at.restart();
  if (key == 's') at.toggleFrameSaving();
}
```

CORE API

Method	Returns	Description
loadConfiguration(file)	this	Load JSON config
setConfiguration(cfg)	this	Set Configuration object
getConfiguration()	Configuration	Get current config
initialize()	this	Build grid & start
render()	void	Draw full-window grid
renderAt(x,y,w,h)	void	Draw into rectangle
restart()	void	Reset animation timer
setAutoRender(bool)	this	Enable/disable auto draw
isAutoRender()	boolean	Check auto-render state
isRunning()	boolean	Animation still playing?
getProgress()	float	0.0 → 1.0 progress
getFrameCount()	int	Current frame index
getTotalFrames()	int	Total frames in anim
toggleFrameSaving()	this	Toggle frame export
setFrameSaving(bool)	this	Set frame export on/off
getFramesDirectory()	String	Output folder path
dispose()	void	Clean up resources

CONFIG.JSON SCHEMA

```
{
  "canvas": { "width": 1080, "height": 1080 },
  "animation": {
    "duration": 18,
    "fps": 30,
    "changeTime": 6000,
    "secondChangeTime": 12000,
    "fadeDuration": 2000,
    "character": "A", // or a word: "TYPE"
    "textScale": 0.8,
    "waveSpeed": 1.0, "waveAngle": 45,
    "waveType": "SINE",
    "waveMultiplierMin": 0.0, "waveMultiplierMax": 2.0,
    "saveFrames": false
  },
  "grid": {
    "initialTilesX": 16, "initialTilesY": 16,
    "changedTilesX": 8,  "changedTilesY": 8,
    "finalTilesX": 4,    "finalTilesY": 4
  },
  "colors": {
    "hueMin": 0, "hueMax": 0,
    "saturationMin": 0, "saturationMax": 0,
    "brightnessMin": 50, "brightnessMax": 255,
    "waveAmplitudeMin": -200, "waveAmplitudeMax": 200,
    "backgroundR": 0, "backgroundG": 0, "backgroundB": 0
  },
  "cellBorder": {
    "sides": 0,
    "r": 255, "g": 255, "b": 255,
    "weight": 1.0, "colorMode": 0
  },
  "cellMotion": {
    "style": "perlin",
    "radius": 12,
    "speed": 1.0
  },
  "glyphOutline": {
    "style": "none",
    "r": 255, "g": 255, "b": 255,
    "weight": 1.5,
    "dashLength": 8.0,
    "gapLength": 4.0
  },
  "gridStripMotion": { // v0.2.6
    "axis": "row", // "row" · "column" · "both"
    "amplitude": 0.3,
    "phaseStep": 0.4,
    "rowWaveType": "SINE",
    "columnWaveType": "SINE",
    "rowSpeed": 1.0,
    "columnSpeed": 1.0,
    "enabled": true
  }
}
```

WAVE & STYLE

Wave presets

```
at.setWaveType(WavePresets.Type.SINE);
// SINE · TANGENT · SQUARE · TRIANGLE · SAWTOOTH
at.setWaveType("SQUARE"); // string overload
// or via config.json: "waveType": "TANGENT"
```

Custom wave function (lambda)

```
at.setWaveFunction(
  (x, y, time, angle) -> {
    return (float)Math.sin(
      x * 0.1 + time
    );
  }
);
```

Design system presets

```
at.useDesignSystem(
  DesignSystem.SystemType.SWISS
);
// SWISS · BAUHAUS · JAPANESE
// BRUTALIST · MINIMAL
```

Vibe coding — natural language

```
at.setVibe("calm ocean waves");
at.setVibe("chaos energy");
at.setVibe("geometric minimal");
```

Glyph motion (v0.2.1 — v0.2.4)

```
// Perlin noise wander
cfg.setCellMotion(new PerlinMotion(12, 1.0f));
// Circular orbital (CW / CCW)
cfg.setCellMotion(new CircularMotion(12, 1.0f, true));
// Lissajous figure-8 (1:2) or knot (3:2)
cfg.setCellMotion(new LissajousMotion(10, 1.0f));
// Spring-damped glyphs
cfg.setCellMotion(new SpringMotion(0.35f, 0.12f));
// Gravity + bounce  (kick() for impulse)
cfg.setCellMotion(new GravityMotion(0.4f, 0.7f));
// Mouse repel/attract field
cfg.setCellMotion(new MagneticMotion(this));
// ── v0.2.3 ──────────────────────────────
// Ripple: click-triggered concentric rings
RippleMotion r = new RippleMotion(14, 1.0f);
r.setTileGrid(width, height, tilesX, tilesY);
r.trigger(mouseX, mouseY); // in mousePressed()
// FlowField: Perlin vector field
cfg.setCellMotion(new FlowFieldMotion(12, 1.0f));
// Orbital: constellation orbit patterns
cfg.setCellMotion(new OrbitalMotion(10, 1.0f));
// Disable
cfg.setCellMotion(null);
// ── JSON alternative (v0.2.4) ──────────
// Any style above can be set in config.json:
// "cellMotion": { "style": "gravity",
//   "radius": 14, "speed": 1.0,
//   "gravity": 0.5, "restitution": 0.8 }
```

GRID STAGES & TIMING

Three-stage grid progression

Stage	Grid Size	Trigger	Default
1 Initial	initialTilesX × Y	0 ms	16 × 16
2 Changed	changedTilesX × Y	changeTime	8 × 8 @ 6 s
3 Final	finalTilesX × Y	secondChangeTime	4 × 4 @ 12 s

`fadeDuration` Cross-fade duration between stages (default 2000 ms)

`secondChangeTime` Set to 0 to disable the third stage

Reading grid state at runtime

```
Configuration cfg = at.getConfiguration();

cfg.getInitialTilesX();   // stage 1 cols
cfg.getInitialTilesY();   // stage 1 rows
cfg.getChangedTilesX();   // stage 2 cols
cfg.getChangedTilesY();   // stage 2 rows
cfg.getFinalTilesX();     // stage 3 cols
cfg.getFinalTilesY();     // stage 3 rows

cfg.getChangeTime();      // ms → stage 2
cfg.getSecondChangeTime();// ms → stage 3
cfg.getFadeDuration();    // crossfade ms
```

Determining current stage manually

```
int elapsed = millis() - startTime;
int cols, rows;

if (elapsed > cfg.getSecondChangeTime()) {
  cols = cfg.getFinalTilesX();
  rows = cfg.getFinalTilesY();
} else if (elapsed > cfg.getChangeTime()) {
  cols = cfg.getChangedTilesX();
  rows = cfg.getChangedTilesY();
} else {
  cols = cfg.getInitialTilesX();
  rows = cfg.getInitialTilesY();
}
```

CONFIGURATION GETTERS

Method	Type	Default
getCanvasWidth()	int	1080
getCanvasHeight()	int	1080
getAnimationDuration()	int	18
getAnimationFPS()	int	30
getChangeTime()	int	6000
getSecondChangeTime()	int	12000
getFadeDuration()	int	2000
getCharacter()	String	"A"
getTextScale()	float	0.8
getWaveSpeed()	float	1.0
getWaveAngle()	float	45.0
getWaveType()	String	"SINE"
getWaveMultiplierMin()	float	0.0
getWaveMultiplierMax()	float	2.0
getInitialTilesX/Y()	int	16
getChangedTilesX/Y()	int	8
getFinalTilesX/Y()	int	4
getHueMin/Max()	float	0 / 0
getSaturationMin/Max()	float	0 / 0
getBrightnessMin/Max()	float	50 / 255
getWaveAmplitudeMin/Max()	float	-200 / 200
getBackgroundRed/Green/Blue()	int	0
isSaveFrames()	boolean	false
getCellBorderSides()	int	0
getCellBorderRed/Green/Blue()	int	255
getCellBorderWeight()	float	1.0
getCellBorderColorMode()	int	0 = static, 1 = wave
getGlyphOutlineStyle()	int	0=none, 1=solid, 2=dashed, 3=dashedOnly
getGlyphOutlineRed/Green/Blue()	int	255
getGlyphOutlineWeight()	float	1.5
getGlyphOutlineDashLength()	float	8.0 (dashed only)
getGlyphOutlineGapLength()	float	4.0 (dashed only)

RENDER MODES

Auto Default — library clears & draws each frame

```
at.initialize();
// draw() is empty — auto-render handles it
```

Manual You control what goes on the canvas

```
at.setAutoRender(false);
at.initialize();

void draw() {
  background(0);
  // … your drawing code …
  at.renderAt(0, 0, width, height);
}
```

Multi Several instances side by side

```
AlgorithmicTypography a, b;
void setup() {
  size(1600, 800);
  a = new AlgorithmicTypography(this);
  b = new AlgorithmicTypography(this);
  a.setAutoRender(false); b.setAutoRender(false);
  a.loadConfiguration("a.json"); b.loadConfiguration("b.json");
  a.initialize(); b.initialize();
}
void draw() {
  a.renderAt(0,0,width/2,height);
  b.renderAt(width/2,0,width/2,height);
}
```

EXPORT & SVG

Save frames as PNG sequence

```
// Enable in config.json
"saveFrames": true

// Or toggle at runtime
at.toggleFrameSaving();
at.setFrameSaving(true);

// Check output directory
println(at.getFramesDirectory());
```

HSB Colours render correctly in PDF & SVG — the library switches colorMode(HSB) before every draw call and restores RGB after. Hue-driven palettes look identical on screen and in export.

Save vector SVG

```
import processing.svg.*;

void keyPressed() {
  if (key == 's') {
    beginRecord(SVG, "output.svg");
    at.render();
    endRecord();
    println("SVG saved!");
  }
}
```

Common keyboard shortcuts

Key	Action
R	Restart animation
S	Toggle frame saving / save SVG

BACKGROUND IMAGE OVERLAY

Render typography on top of a photograph — place the image in the sketch's `data/` folder.

```
AlgorithmicTypography at;
PImage bg;

void setup() {
  size(800, 800);
  bg = loadImage("background.png");
  bg.resize(width, height);

  at = new AlgorithmicTypography(this);
  at.loadConfiguration("config.json");
  at.setAutoRender(false);
  at.initialize();
}

void draw() {
  image(bg, 0, 0);          // draw photo first
  at.renderAt(0, 0, width, height); // overlay grid
}
```

PER-CELL PHOTOGRAPHS (MULTIPHOTO)

Different photo behind each grid cell. Place photo-01.png … photo-16.png in `data/`

```
// setup(): load photos into PImage[] photos
for (int i = 1; i <= 16; i++) {
  PImage img = loadImage("photo-"+nf(i,2)+".png");
  if (img != null && img.width > 0)
    photos[photoCount++] = img;
}
```

```
// draw(): tile photos, then overlay glyphs
float cw = (float)width/cols, ch = (float)height/rows;
for (int gx=0; gx<cols; gx++)
 for (int gy=0; gy<rows; gy++) {
   int idx = (gy*cols+gx) % photoCount;
   image(photos[idx], gx*cw, gy*ch, cw, ch);
 }
at.renderAt(0, 0, width, height);
```

See `MultiPhoto` example for centre-crop & 3-stage tracking.

EXAMPLES GALLERY — 28 SKETCHES

AudioReactive
Audio input drives wave parameters in real time.

BackgroundImage
Typography overlay on a single background photograph.

BasicGrid
Minimal grid with JSON config — the "hello world" sketch.

CulturalStyles
Switch between built-in design systems (Swiss, Bauhaus …). *Work in progress.*

CustomFont
Load and render a custom .ttf / .otf font.

CustomWave
Provide a lambda for a custom wave function.

GlyphBoolean
Dual-mode boolean showcase. Mode A (Letter×Letter): 1/2/3 cycle union/intersect/subtract on curated pairs. Mode B (Shape×Letter): geometric shapes composited with a letterform; mode 4 adds a band cutter. TAB switches modes; SPACE cycles pairs/shapes; S saves PNG. (v0.2.5)

GlyphOutline
Config-driven glyph outlines via glyphOutline JSON block. O cycles 4 styles: None → Solid → Dashed → Dashed Only (fill suppressed); left/right arrows cycle 16 test chars; S saves PNG. (v0.2.5)

GlyphDesign
fillWithPoints, distributeAlongOutline, getOuterContour / getInnerContours — geometric glyph analysis (v0.2.2).

GlyphMorph
Dedicated showcase for interpolateTo(): large central morph, five-thumbnail timeline strip, four styles (Outline / Filled / Dots / Dual), mouse-X scrub (v0.2.3).

GlyphDynamics
Particle-based physics simulation on glyph outlines.

GlyphPath
11 display modes: filled, points, deformed, contours, tiled grid, interior fill, outer contour, outer+inner, hatch fill, morph, path particle (v0.2.3).

GlyphWander
Cycle 9 motion modes: None, Perlin, Circular CW/CCW, Lissajous, Spring, Ripple, FlowField, Orbital. Click triggers ripple ring. (Gravity & Magnetic have dedicated standalone examples.)

GridStripWave
Row/column strip-wave displacement via GridStripMotion. Cycles ROW → COLUMN → BOTH axes; W cycles 6 wave types; +/− adjusts amplitude; A adjusts phase step; S saves PNG. (v0.2.6)

GravityDynamics
Glyphs fall and bounce with live sliders; SPACE triggers an upward kick impulse.

LiveControls
Adjust parameters live via GUI sliders, keyboard, and OSC. Includes border weight slider; B cycles sides, V toggles wave-reactive border colour.

MagneticDynamics
Mouse-driven repel/attract field; SPACE toggles polarity; 3 presets.

MultiPhoto
Different photograph behind each grid cell with stage tracking.

MultipleSystems
Run several AlgorithmicTypography instances side by side.

PerformanceMode
Optimised rendering path for high-resolution grids. *Work in progress.*

ProgrammaticConfig
Build Configuration in code with the Builder API. Randomises border sides and colour; B toggles wave-reactive border mode.

RandomASCII
Random ASCII characters in each cell.

SaveSVG
Export the current frame as a vector SVG file.

TextDrivenAnimation
Typography driven by external text file content.

TextOnPath
Mode 1: text string on outer contour via textOnPath(). Mode 2: 48 tangent-oriented ornaments via getTangent(). SPACE/T cycle chars & text, S saves. (v0.2.5)

TrailEffect
Motion trail / ghosting across all 6 CellMotion types.

VibeCoding
All 26 vibe keywords across 7 groups; ←/→ cycle, SPACE random compound.

WaveAngle
Explore wave propagation angle with JSON config. B key toggles horizontal rule borders (top+bottom sides).

GLYPHEXTRACTOR API (V0.2.2 – V0.2.3)

Designer-oriented glyph geometry access

```
import algorithmic.typography.render.GlyphExtractor;
import java.util.List;

GlyphExtractor g = new GlyphExtractor(this, "Helvetica", 72);
g.setFlatness(0.3); // curve resolution

// Fluent GlyphBuilder (v0.2.3)
GlyphExtractor.of('A', 400)
  .fillPoints(500)
  .hatch(45, 8)
  .colourByPosition()
  .hueRange(180, 270)
  .draw(this, x, y);
```

Method	Returns	Notes
extractChar(ch, size)	PShape	filled shape
extractDeformed(ch, sz, …)	PShape	wave-distorted
getContours(ch, size)	List<PVector[]>	all contours
getOuterContour(ch, size)	PVector[]	largest area
getInnerContours(ch, size)	List<PVector[]>	counter-forms
fillWithPoints(ch, size, n)	PVector[]	interior scatter
distributeAlongOutline(ch, sz, n)	PVector[]	arc-length spaced
getBounds(ch, size)	float[4]	x,y,w,h
fillWithLines(ch, sz, ang, gap)	float[][]	hatch segments
offsetOutline(ch, size, dist)	PVector[]	inline / outline
interpolateTo(c1, c2, sz, t)	PVector[]	morph at t∈[0,1]
getMedialAxis(ch, size, n)	PVector[]	spine approx.
sampleAlongPath(ch, size, t)	PVector	point at arc t
getBoundingContour(chs[], sz, f)	PVector[]	union outline
of(ch, size) → GlyphBuilder	GlyphBuilder	fluent API
centerOf(str, sz, cx, cy)	PVector	word centering offset
getDashedOutline(str, sz, dash, gap)	float[][]	word dashed outline

v0.2.3 rows highlighted in accent colour.
See **Page 6** for v0.2.5 additions: boolean ops, path utilities & Type DNA.
Bottom two rows: v0.2.5 word-support String overloads — pass a word wherever a char was accepted.

PACKAGE STRUCTURE

Package	Key classes
algorithmic.typography	AlgorithmicTypography, Configuration, HeadlessRenderer
…typography.core	WaveEngine, WavePresets, CellMotion, all Motion types, TemporalTrail
…typography.render	GridRenderer, GlyphExtractor, GlyphBuilder, TypeDNAProfile, GlyphPhysics, ExportController, VectorExporter
…typography.audio	AudioBridge (FFT, beat detection)
…typography.style	ColorPalette, Serendipity
…typography.system	DesignSystem, VibePreset
…typography.text	RitaBridge (RiTa integration)
…typography.ui	ControlPanel, Slider, ProgressBar, OSCBridge
…typography.ml	AIWaveFunction
…typography.net	WebSocketServer

CELLMOTION TYPES

Class	Since	Description
CircularMotion	0.2.1	CW / CCW orbital
PerlinMotion	0.2.1	noise wandering
LissajousMotion	0.2.1	figure-8 / knot
SpringMotion	0.2.1	spring-damped chase
GravityMotion	0.2.1	fall + bounce + kick()
MagneticMotion	0.2.2	mouse repel / attract
RippleMotion	0.2.3	concentric click rings
FlowFieldMotion	0.2.3	Perlin vector field
OrbitalMotion	0.2.3	constellation orbit

BOOLEAN AREA OPERATIONS (V0.2.5)

AWT Area boolean algebra on native font outlines. Both PShape (draw with shape()) and PVector[] (physics / custom render) return variants.

```
import algorithmic.typography.render.*;

GlyphExtractor g = new GlyphExtractor(this, "Helvetica", 72);

// PShape overloads — ready to draw with shape()
PShape u = g.union('O', 'C', 400);
PShape i = g.intersect('B', 'E', 400);
PShape s = g.subtract('A', 'V', 400);
shape(u, x, y);

// PVector[] overloads — for physics / custom draw
PVector[] uc = g.getUnionContour('O', 'C', 400);
PVector[] ic = g.getIntersectContour('B', 'E', 400);
PVector[] sc = g.getSubtractContour('A', 'V', 400);

// Arc-length-resampled dashed outline
float[][] dash = g.getDashedOutline('O', 400, 10, 5);
for (float[] seg : dash)
  line(seg[0], seg[1], seg[2], seg[3]);
```

Method	Returns	Description
union(c1,c2,sz)	PShape	merged outline
getUnionContour(c1,c2,sz)	PVector[]	merged contour p
intersect(c1,c2,sz)	PShape	overlap region
getIntersectContour(c1,c2,sz)	PVector[]	overlap contour pts
subtract(c1,c2,sz)	PShape	c1 minus c2
getSubtractContour(c1,c2,sz)	PVector[]	knockout contour
getDashedOutline(ch,sz,dash,gap)	float[][]	{x1,y1,x2,y2}/se per-contour
getDashedOutline(str,sz,dash,gap)	float[][]	word variant; pe contour

Config-driven: add "glyphOutline": {"style":"dashedOnly", …} to config.json — AT renders outlines automatically. Four styles: OUTLINE_NONE, OUTLINE_SOLID, OUTLINE_DASHED, OUTLINE_DASHED_ONLY (fill suppressed). See `GlyphOutline`: O cycles all 4; ←/→=char; S=save.

PATH UTILITIES (V0.2.5)

```
// Unit tangent direction at arc-length t ∈ [0,1]
PVector tan = g.getTangent('A', 400, 0.25f);
float angle   = atan2(tan.y, tan.x);

// Orient 48 ornaments along an outline
PVector[] path = g.getOuterContour('O', 600);
for (int k = 0; k < 48; k++) {
  float t   = (float) k / 48;
  PVector pos = /* samplePath(path, t) */;
  PVector dir = g.getTangent('O', 600, t);
  pushMatrix(); translate(pos.x, pos.y);
  rotate(atan2(dir.y, dir.x));
  text(">", 0, 0); popMatrix();
}

// Extra tessellation density for deformation
PVector[] dense = g.subdivide('O', 400, 6);

// Lay a string along any contour
PVector[] outer = g.getOuterContour('O', 600);
g.textOnPath("Around the letterform", outer, 24);
```

Method	Returns	Notes
getTangent(ch, size, t)	PVector	unit dir at arc t∈[0,1]
subdivide(ch, size, n)	PVector[]	n pts per segment
textOnPath(text, path, sz)	PShape	glyph group on contour
textOnPath(text, path, sz, spacing)	PShape	+ extra px between chars

See `TextOnPath`: mode 1 text on outer contour; mode 2 tangent-oriented ornaments; SPACE cycles chars, T cycles text, S saves.

TYPE DNA — TYPEDNAPROFILE & APPLYTYPEDNA (V0.2.5)

Font outlines drive the animation. Switch typefaces and the animation changes character automatically.

```
import algorithmic.typography.render.*;

GlyphExtractor g =
  new GlyphExtractor(this, "Garamond", 600);

// Individual metrics
float   axis   = g.getStressAxis('A', 600);  // °
PVector centre = g.getOpticalCentroid('A', 600);
float   ratio  = g.getCounterRatio('O', 600);  // 0–1
float   weight = g.getStrokeWeight('A', 600); // px

// Aggregate into a reusable profile
TypeDNAProfile dna =
  g.buildTypeDNAProfile('A', 600);

// One-call preset — returns this for chaining
at.applyTypeDNA(dna).initialize();

// JSON round-trip — save & share presets
saveJSONObject(dna.toJSON(), "garamond.json");
TypeDNAProfile p =
  TypeDNAProfile.fromJSON(
    loadJSONObject("garamond.json"));
at.applyTypeDNA(p).initialize();
```

applyTypeDNA maps the profile:

DNA field	Config parameter set
stressAxis	waveAngle
counterRatio	waveAmplitudeRange
strokeWeight	brightnessRange

Method	Returns	Use as…
getStressAxis(ch,sz)	float °	waveAngle
getOpticalCentroid(ch,sz)	PVector	orbit / pivot origin
getCounterRatio(ch,sz)	float 0–1	amplitude / breathing
getStrokeWeight(ch,sz)	float px	brightness / mass
buildTypeDNAProfile(ch,sz)	TypeDNAProfile	full fingerprint

GRIDSTRIPMOTION (V0.2.6)

Shifts entire rows or columns as rigid strips driven by a wave. Composes with CellMotion — strip slides + per-glyph orbits work simultaneously.

```
// Via config.json (recommended)
// "gridStripMotion": { "axis": "row",
//   "amplitude": 0.3, "phaseStep": 0.4 }

// Or programmatically
GridStripMotion strip = new GridStripMotion();
strip.setAxis(GridStripMotion.ROW);   // ROW · COLUMN · BOTH
strip.setAmplitude(0.3f);    // fraction of tile size
strip.setPhaseStep(0.4f);    // radians between strips
strip.setRowWaveType("SINE"); // per-axis wave override
strip.setColumnWaveType("PERLIN");
strip.setRowSpeed(1.0f);
strip.setColumnSpeed(-1.0f); // negative = reverse
at.setGridStripMotion(strip);
```

Method	Notes
setAxis(int)	ROW=0, COLUMN=1, BOTH=2
setAmplitude(float)	0–1 fraction of tile size
setPhaseStep(float)	radians; more = more cycles visible
setRowWaveType(String)	SINE · SQUARE · TRIANGLE ·
setColumnWaveType(String)	SAWTOOTH · TANGENT · PERLIN
setRowSpeed(float)	negative reverses direction
setColumnSpeed(float)	counter-travel when opposite sign
setEnabled(boolean)	toggle without destroying state
at.setGridStripMotion(gsm)	attach to AlgorithmicTypography

PERLINVERTEXMOTION (V0.2.6)

Displaces each glyph outline vertex with independent X/Y Perlin fields. Works on arrays from GlyphExtractor.

```
GlyphExtractor ge = new GlyphExtractor(this, "Helvetica", 72);
PVector[] outline = ge.getOuterContour('A', 200);

PerlinVertexMotion pvm = new PerlinVertexMotion(this);
pvm.setAmplitude(6.0f);      // px displacement
pvm.setSpatialScale(0.03f); // field frequency
pvm.setTimeSpeed(0.015f);  // evolution rate/frame

// In draw():
PVector[] warped = pvm.deform(outline, frameCount);
beginShape();
for (PVector v : warped) vertex(v.x, v.y);
endShape(CLOSE);

// Multi-contour shortcut
PVector[][] all = ge.getContours('B', 200);
PVector[][] w   = pvm.deformContours(all, frameCount);
```

Preset ranges of note:

Feel	amplitude	spatialScale	timeSpeed
Subtle	2.0	0.05	0.008
Organic	6.0	0.03	0.015
Chaotic	14.0	0.012	0.04

MAGNETICMOTION & RIPPLEMOTION NAMED PRESETS (V0.2.6)

Replace raw parameter tables with a single setPreset(int) call.

```
// MagneticMotion — 4 intensity levels
MagneticMotion mag = new MagneticMotion(this);
mag.setPreset(MagneticMotion.GENTLE);   // drift away
mag.setPreset(MagneticMotion.MODERATE); // balanced
mag.setPreset(MagneticMotion.STRONG);   // forceful
mag.setPreset(MagneticMotion.SNAPPING); // instant snap

// Normalised strength (0 = off, 1 = preset max)
mag.setStrengthNormalized(0.6f);

// RippleMotion — 3 intensity levels
RippleMotion rip = new RippleMotion(this);
rip.setPreset(RippleMotion.GENTLE);
rip.setPreset(RippleMotion.MODERATE);
rip.setPreset(RippleMotion.STRONG);

// Normalised speed (0 = frozen, 1 = preset max)
rip.setExpandSpeedNormalized(0.5f);
```

AUDIOBRIDGE SEMANTIC MAPPING (V0.2.6)

Three named intensity constants replace manual min/max ranges. Each band has pre-tuned standard ranges.

```
AudioBridge audio = new AudioBridge(this);
audio.initialize();

// Semantic mapping — intensity constants:
// SUBTLE (0) · EXPRESSIVE (1) · FULL (2)
audio.mapBassTo(
  config::setWaveSpeed, AudioBridge.EXPRESSIVE);
  // bass SUBTLE=[0.8,1.5]  EXP=[0.3,3.0]  FULL=[0,6]

audio.mapMidTo(
  config::setSaturationMax, AudioBridge.SUBTLE);
  // mid  SUBTLE=[70,140]   EXP=[30,200]   FULL=[0,255]

audio.mapTrebleTo(
  config::setBrightnessMax, AudioBridge.FULL);
  // treb SUBTLE=[150,220]  EXP=[80,240]   FULL=[50,255]

audio.mapOverallTo(
  config::setWaveMultiplierMax, AudioBridge.EXPRESSIVE);
  // ovrl SUBTLE=[0.5,1.5]  EXP=[0.2,3.0]  FULL=[0,6]

// In draw():
audio.update();
```

QUICK TIPS (1–6)

#	Tip
1	Always call initialize() after loading or setting a configuration.
2	Use setAutoRender(false) whenever you draw backgrounds or composite.
3	renderAt(x, y, w, h) places the grid anywhere — split layouts, insets, etc.
4	Track your own startTime = millis() — the library's start time is private.
5	Call restart() + reset startTime together when restarting.
6	Set secondChangeTime to 0 in JSON to use only two grid stages.

QUICK TIPS (7–12)

#	Tip
7	Colours are HSB: hue 0–360, saturation 0–255, brightness 0–255.
8	Place images and JSON files in the sketch's data/ folder.
9	Builder pattern: new Configuration.Builder().initialTiles(8,8).build()
10	Javadoc is in reference/ — open index.html in a browser.
11	RippleMotion.trigger(x,y) must be called from mousePressed(); call setTileGrid() once in setup().
12	All GlyphExtractor methods have String overloads — pass a word (e.g. "TYPE") wherever a single char was accepted.